



Data: 26/11/2025

Progetto: OikoNaos	Versione: 1.6
Documento: System Design	Data: 26/11/2025

Coordinatore del progetto:

Nome	Matricola
Luigi Potestà	0512120076

Partecipanti:

Nome	Matricola
Luigi Potestà	0512120076
Maria Antonietta Ruggiero	0512121126
Giulia Buonafine	0512119695
Mario Sinopoli	0512119113

Scritto da:	Giulia Buonafine
--------------------	------------------

Revision History

Data	Versione	Descrizione	Autore
15/11/2025	1.0	Prima stesura e definizione di Design Goals e Glossario	Mario Sinopoli
18/11/2025	1.1	Stesura delle sezioni Architettura corrente e Architettura proposta (Overview), inclusa introduzione dell'architettura 3-layer (Presentation, Application, Persistence)	Giulia Buonafine
20/11/2025	1.2	Prima bozza del documento SDD OikoNaos, con stesura iniziale di 'Gestione dati persistenti' e 'Controllo degli accessi e sicurezza'	Maria Antonietta Ruggiero
21/11/2025	1.3	Dettaglio della 'Decomposizione' in sottosistemi per i livelli View e Controller; integrazione dei componenti di 'Gestione Notifiche' e 'Gestione Risorse'	Luigi Potestà
22/11/2025	1.4	Ridefinizione sezioni 5.2 e 5.3 con aggiunta di diagrammi di decomposizione e mapping H/S; stesura delle sezioni 'Controllo flusso globale' e 'Boundary Conditions'	Giulia Buonafine

24/11/2025	1.5	Finalizzazione della sezione ‘Glossario dei Servizi dei Sottosistemi’ (Sez. 6), verificando la coerenza con i casi d'uso	Mario Sinopoli
26/11/2025	1.6	Miglioramento della formattazione	Luigi Potestà

Indice

1. Purpose	5
2. Audience	5
3. Introduction	6
3.1. Purpose of the system.....	6
3.2. Design goals	7
3.3. Definitions, acronyms, abbreviations.....	8
3.4. References	8
3.5. Overview	8
4. Current Software Architecture	9
5. Proposed Software Architecture	9
5.1. Overview	9
5.2. Subsystem decomposition	10
5.3. Hardware/Software mapping.....	13
5.4. Persistent data management	14
5.5. Access control and security	15
5.6. Global software control.....	17
5.7. Boundary conditions	18
6. Subsystem Service Glossary.....	19

1. Purpose

Il *System Design Document* (SDD) di OikoNaos definisce l'architettura tecnica della piattaforma e traduce i requisiti formalizzati nel *Requirements Analysis Document* (RAD) in un insieme coerente di decisioni progettuali.

Il documento descrive in modo completo e strutturato gli obiettivi di design, l'organizzazione del sistema in sottosistemi, le componenti hardware e software adottate e i meccanismi fondamentali che regolano il funzionamento della piattaforma.

In particolare, l'SDD illustra la struttura complessiva del sistema e la sua decomposizione in moduli funzionali; presenta la modellazione statica tramite UML (class diagram, package diagram) e la modellazione dinamica attraverso diagrammi di sequenza e diagrammi di attività; definisce l'architettura di deployment e la mappatura hardware/software; specifica le strategie di gestione e persistenza dei dati; descrive i meccanismi di accesso, autorizzazione, sicurezza, comunicazione e interfaccia tra i diversi componenti. Tali elementi garantiscono una visione unificata e rigorosa dell'architettura di OikoNaos, necessaria per supportare una realizzazione implementativa solida e coerente.

Il documento svolge inoltre un ruolo centrale nel coordinamento dello sviluppo: chiarisce le responsabilità dei vari sottosistemi, stabilisce le interfacce e costituisce un riferimento costante durante l'intero ciclo di vita del progetto, agevolando il riesame delle scelte architetturali e la loro eventuale evoluzione. L'SDD assicura che l'architettura proposta sia perfettamente allineata al *Problem Statement* e pienamente conforme ai requisiti funzionali e non funzionali definiti nel RAD, diventando così uno strumento fondamentale per la qualità, la coerenza e la sostenibilità dell'intero sistema.

2. Audience

Il presente *System Design Document* (SDD) è rivolto a tutte le figure coinvolte nella progettazione, nello sviluppo e nella supervisione del sistema OikoNaos. Esso rappresenta un riferimento tecnico centralizzato che garantisce coerenza, chiarezza e allineamento tra i diversi attori impegnati nel progetto.

Il documento è destinato innanzitutto al **Project Management**, responsabile della pianificazione complessiva e del coordinamento delle attività. Il SDD consente loro di monitorare la corrispondenza tra obiettivi, requisiti formali e scelte architetturali, assicurando che il sistema evolva in conformità con quanto definito nel RAD e nel Problem Statement.

È rivolto poi ai **System Architects**, incaricati della progettazione dell'architettura complessiva. Per questa figura, il SDD raccoglie in modo strutturato la descrizione dei sottosistemi, la loro organizzazione e le relative dipendenze, includendo i modelli UML (class diagram, sequence diagram, package diagram, activity diagram) e le scelte tecnologiche fondamentali come Java 17, Apache

Tomcat 10.1 e MySQL 8. Il documento fornisce così le informazioni necessarie per valutare, validare e ottimizzare la struttura del sistema.

Il SDD è rivolto anche agli **sviluppatori del sistema**, cioè a chi si occupa di realizzare le diverse parti dell'applicazione e del database. Il documento costituisce una guida chiara sulle funzionalità di ogni componente, quali devono essere implementate e quali regole devono essere rispettate. Grazie all'SDD, gli sviluppatori possono lavorare in modo coordinato, mantenere coerenza tra le varie parti del sistema e integrare correttamente i moduli, seguendo i modelli UML e i meccanismi di sicurezza e autorizzazione previsti dal progetto. Infine, il documento può essere consultato dagli **Stakeholder Tecnici**, quali l'Azienda Coordinatrice che, pur non partecipando direttamente allo sviluppo, necessita di verificare le scelte strutturali alla base della piattaforma. Per queste figure, l'SDD offre una visione chiara sulla scalabilità, affidabilità e sicurezza dell'architettura, includendo conformità a linee guida come GDPR, trasmissione sicura via HTTPS e gestione crittografata delle password.

Nel complesso, l'SDD funge da strumento di comunicazione e coordinamento tra tutte le parti coinvolte, supportando la progettazione, lo sviluppo e la manutenzione del sistema OikoNaos in tutte le fasi del suo ciclo di vita.

3. Introduction

3.1. Purpose of the system

Lo scopo del sistema OikoNaos è fornire un'infrastruttura digitale centralizzata per la gestione efficiente e trasparente delle comunità di co-housing supervisionate da un'Azienda Coordinatrice. La piattaforma nasce per sostituire l'attuale gestione frammentata, basata su e-mail, comunicazioni informali e strumenti non integrati, introducendo un ambiente unico e strutturato che supporti tutte le principali attività quotidiane dei coinquilini. Il sistema permette infatti ai residenti di effettuare prenotazioni degli spazi condivisi, segnalare problemi tramite ticket, partecipare agli eventi, richiedere risorse comuni e monitorare lo stato dei pagamenti, semplificando le interazioni e favorendo la partecipazione alla vita comunitaria.

Parallelamente, i supervisori delle comunità dispongono di funzionalità dedicate per monitorare le attività dei residenti, gestire e validare ticket e richieste, controllare le prenotazioni e verificare il corretto svolgimento dei processi interni. A livello superiore, l'Azienda Coordinatrice può assegnare i codici identificativi necessari alla registrazione, gestire i supervisori e generare report amministrativi utili al controllo operativo.

In questo modo, OikoNaos integra in un'unica piattaforma digitale sia le esigenze quotidiane dei coinquilini sia i compiti amministrativi dell'organizzazione che supervisiona i complessi abitativi. L'obiettivo complessivo è garantire continuità nei servizi, sicurezza e integrità dei dati, uniformità nei processi e un modello di gestione moderno, scalabile e coerente con le necessità dell'intero ecosistema del co-housing.

3.2. Design goals

L'architettura di OikoNaos è guidata da un insieme di design goals derivati direttamente dai requisiti funzionali e non funzionali del *Requirements Analysis Document* e orientati alla realizzazione di un sistema affidabile, scalabile e coerente con le esigenze operative delle comunità di co-housing. Tali obiettivi definiscono il quadro metodologico entro cui è stata sviluppata la progettazione del sistema, garantendo tracciabilità e aderenza alle specifiche.

- **DG01 Modularità:** Prevede la suddivisione del sistema in sottosistemi autonomi e ad alta coesione come autenticazione, gestione comunità, ticketing, risorse condivise e pagamenti. Favorisce manutenzione, riuso e sviluppo parallelo. Questo principio è in linea con i requisiti di configurabilità e modellazione (*RNF10 e RNF17*) e permette un'architettura pulita e facilmente estendibile.
- **DG02 Scalabilità e Performance:** L'architettura deve poter gestire l'aumento del numero di comunità, utenti e risorse mantenendo tempi di risposta adeguati e stabilità del servizio. Questo obiettivo risponde ai requisiti di performance descritti nel RAD (*RNF06 e RNF07*, stabilità e scalabilità del servizio) e consente al sistema di sostenere carichi elevati senza degradazione delle prestazioni.
- **DG03 Sicurezza dei dati,** con particolare attenzione alla conformità al GDPR, costituisce un pilastro dell'architettura. Essa include autenticazione sicura, controllo degli accessi basato sui ruoli, cifratura delle password e protezione delle comunicazioni (in conformità con *RNF22, RNF23 e RNF24*). Questi aspetti sono necessari per proteggere informazioni sensibili, specialmente nelle operazioni di pagamento, gestione profili e amministrazione comunitaria.
- **DG04 Affidabilità e Robustezza:** Il sistema deve garantire correttezza e consistenza dei dati, mantenendo un comportamento stabile anche in presenza di errori o input non validi. Questo obiettivo è legato ai requisiti di accuratezza dei dati (*RNF08*) e alla necessità di assicurare continuità operativa e stabilità del servizio, come richiesto dagli obiettivi generali del RAD.
- **DG05 Supporto alla gestione multilivello:** OikoNaos deve supportare in modo naturale la distinzione tra il livello operativo della comunità e quello dell'Azienda Coordinatrice. L'architettura deve quindi permettere a coinquilini, supervisori di accedere a funzioni differenti, mantenendo una netta separazione tra attività locali e attività amministrative. Questo è reso possibile dal controllo degli accessi basato sui ruoli previsto dal RAD (*RNF24*), che assicura un'organizzazione coerente, sicura e pienamente aderente alla struttura gerarchica del co-housing.
- **DG06 Usabilità:** La piattaforma deve essere semplice da usare e facilmente comprensibile anche per utenti con competenze informatiche limitate. L'interfaccia deve risultare chiara, intuitiva e navigabile senza difficoltà, adattandosi ai diversi dispositivi grazie al design responsive (*RNF16*). L'obiettivo è offrire un'esperienza fluida, che riduca gli errori e permetta agli utenti di compiere rapidamente le operazioni quotidiane.
- **DG07 Portabilità:** Il sistema deve essere compatibile con i principali browser e ambienti server, sfruttando tecnologie standard come Java 17, Tomcat 10.1 e MySQL 8. Questo obiettivo riflette i requisiti di compatibilità e portabilità (in particolare *RNF15*), che richiede l'esecuzione dell'applicazione senza modifiche su Chrome, Firefox, Edge e Safari, insieme ai requisiti di implementazione e configurabilità (*RNF12–RNF14*).

Nel complesso, questi obiettivi definiscono un'architettura solida, scalabile e sicura, strettamente aderente ai requisiti del RAD e progettata per garantire efficienza operativa, continuità del servizio e qualità dell'esperienza utente all'interno delle comunità di co-housing.

3.3. Definitions, acronyms, abbreviations

Termine	Definizione
Azienda Coordinatrice	Organizzazione che sovrintende le comunità di co-housing e ha accesso ai pannelli amministrativi di livello superiore.
Coinquilino	Utente residente nelle strutture di co-housing, dotato di accesso alle funzionalità comunitarie.
Supervisore	Utente con permessi avanzati, responsabile della gestione dei ticket e del monitoraggio della comunità.
Codice Identificativo Speciale	Codice univoco assegnato dall'Azienda Coordinatrice e necessario per la prima registrazione di un coinquilino nel sistema.
RAD	<i>Requirements Analysis Document</i> , documento dei requisiti funzionali e non funzionali.
SDD	<i>System Design Document</i> , documento di progettazione del sistema.
RBAC (Role-Based Access Control)	Meccanismo di controllo degli accessi basato sui ruoli, utilizzato da OikoNaos per differenziare privilegi e funzionalità tra coinquilini, supervisori e azienda coordinatrice.
HTTPS	Protocollo di comunicazione cifrata utilizzato per garantire confidenzialità e integrità nello scambio di informazioni tra client e server.
DBMS	<i>Database Management System</i> – Sistema di gestione del database (MySQL nel caso di OikoNaos).

3.4. References

- **Requisiti funzionali:** Sezione 4.2 del RAD
- **Requisiti non funzionali:** Sezione 4.3 del RAD

3.5. Overview

OikoNaos è una piattaforma web progettata per supportare la gestione operativa, amministrativa e comunitaria delle strutture di co-housing gestite dall'Azienda Coordinatrice. Il sistema integra

funzionalità di registrazione e autenticazione degli utenti, gestione degli spazi comuni, prenotazioni, ticket di manutenzione, pagamenti periodici, comunicazioni interne e supervisione multilivello delle comunità. L'architettura è costruita in modo modulare, con sottosistemi autonomi che cooperano attraverso un insieme coerente di controller applicativi e modelli dati condivisi, come definito nel *Requirements Analysis Document*.

Questo documento di System Design descrive l'architettura logica e fisica del sistema, analizza la decomposizione in sottosistemi, definisce le interfacce tra i componenti e fornisce le decisioni architetturali necessarie per guidare la successiva fase di implementazione.

4. Current Software Architecture

Attualmente, la gestione della comunità di co-housing avviene in modo frammentato e con un livello di digitalizzazione minimo. Le attività quotidiane, come la prenotazione degli spazi comuni, la segnalazione dei guasti o la condivisione di informazioni tra residenti e supervisore, sono svolte attraverso strumenti eterogenei e spesso informali, come e-mail, messaggi privati o comunicazioni verbali. Questo approccio, seppur funzionale in contesti molto piccoli, risulta inefficiente e soggetto a errori quando il numero di residenti e di attività da coordinare aumenta.

L'assenza di un sistema centralizzato produce diverse criticità operative. La gestione manuale delle prenotazioni degli spazi comporta frequenti sovrapposizioni e difficoltà nel verificare in tempo reale la disponibilità delle postazioni. Allo stesso modo, la segnalazione dei guasti avviene senza un meccanismo strutturato, rendendo complicato per il supervisore tenere traccia delle richieste, stabilire priorità o monitorarne la risoluzione.

Dal punto di vista amministrativo, anche la gestione delle spese e delle comunicazioni interne risulta disorganizzata. Le informazioni sui pagamenti vengono spesso registrate tramite fogli di calcolo o comunicazioni isolate, con conseguente rischio di discrepanze o ritardi. La diffusione di annunci ed eventi è affidata a canali non ufficiali, che non garantiscono uniformità né copertura completa tra i residenti.

In sintesi, l'attuale sistema presenta problemi di coordinamento, mancanza di tracciabilità e dispersione delle informazioni. Tali limitazioni ostacolano una gestione fluida ed efficace della **singola comunità** e rendono necessario introdurre una piattaforma unica, strutturata e accessibile, capace di centralizzare i processi e supportare sia i residenti sia il supervisore nelle loro attività quotidiane.

5. Proposed Software Architecture

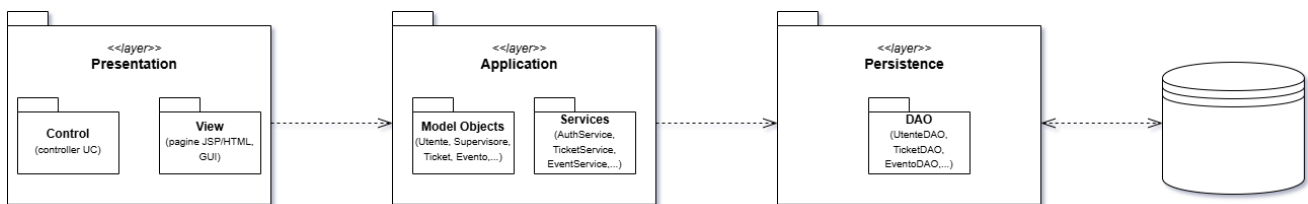
5.1. Overview

L'architettura proposta per OikoNaos è una **web application multilivello** progettata per integrare tutte le attività operative, amministrative e comunitarie delle realtà di co-housing. L'obiettivo è fornire una piattaforma centralizzata in grado di gestire utenti, prenotazioni, ticket, eventi, risorse condivise e

supervisione multilivello, garantendo sicurezza, scalabilità e semplicità di utilizzo. Il sistema si basa su tre livelli principali:

- **Presentazione (client):** interfaccia web accessibile tramite browser, pensata per coinquilini, supervisori e Azienda Coordinatrice.
- **Applicazione (server):** livello che gestisce la logica dei casi d'uso, le regole di business, il controllo dei flussi e l'interazione tra i sottosistemi.
- **Persistenza (database):** livello responsabile della gestione delle informazioni, organizzate in entità coerenti con il modello concettuale definito nel RAD.

Questa struttura, chiara e modulare, consente di garantire facilità di manutenzione, alta coesione dei componenti e possibilità di estendere il sistema nel tempo, mantenendo coerenza e qualità.



5.2. Subsystem decomposition

La decomposizione in sottosistemi segue rigorosamente l'architettura Model–View–Controller, che consente di separare presentazione, logica di business e gestione dei dati. Ogni sottosistema è stato progettato per garantire alta coesione interna e basso accoppiamento secondo i principi di progettazione definiti nei Design Goals. Il sistema è stato suddiviso nei seguenti sottosistemi, organizzati secondo i tre livelli della sua architettura: **View, Controller e Model**.

Nella View (Interfaccia Utente):

- **GUI Coinquilino:** Interfaccia dedicata ai membri della comunità. Permette di effettuare prenotazioni, creare ticket di manutenzione, iscriversi agli eventi, visualizzare pagamenti e consultare la bacheca.
- **GUI Supervisore:** Interfaccia dedicata ai supervisori delle comunità. Consente di gestire ticket, approvare prenotazioni, pubblicare eventi, visualizzare la situazione della comunità e monitorare i residenti.
- **GUI Azienda Coordinatrice:** Interfaccia destinata ai responsabili centrali. Permette di gestire le comunità, assegnare supervisori, generare codici di registrazione e visualizzare report globali.

La View raccoglie input dagli utenti e presenta il risultato dell'elaborazione, senza integrare logica di business.

Nella componente Controller (Logica dei casi d'uso):

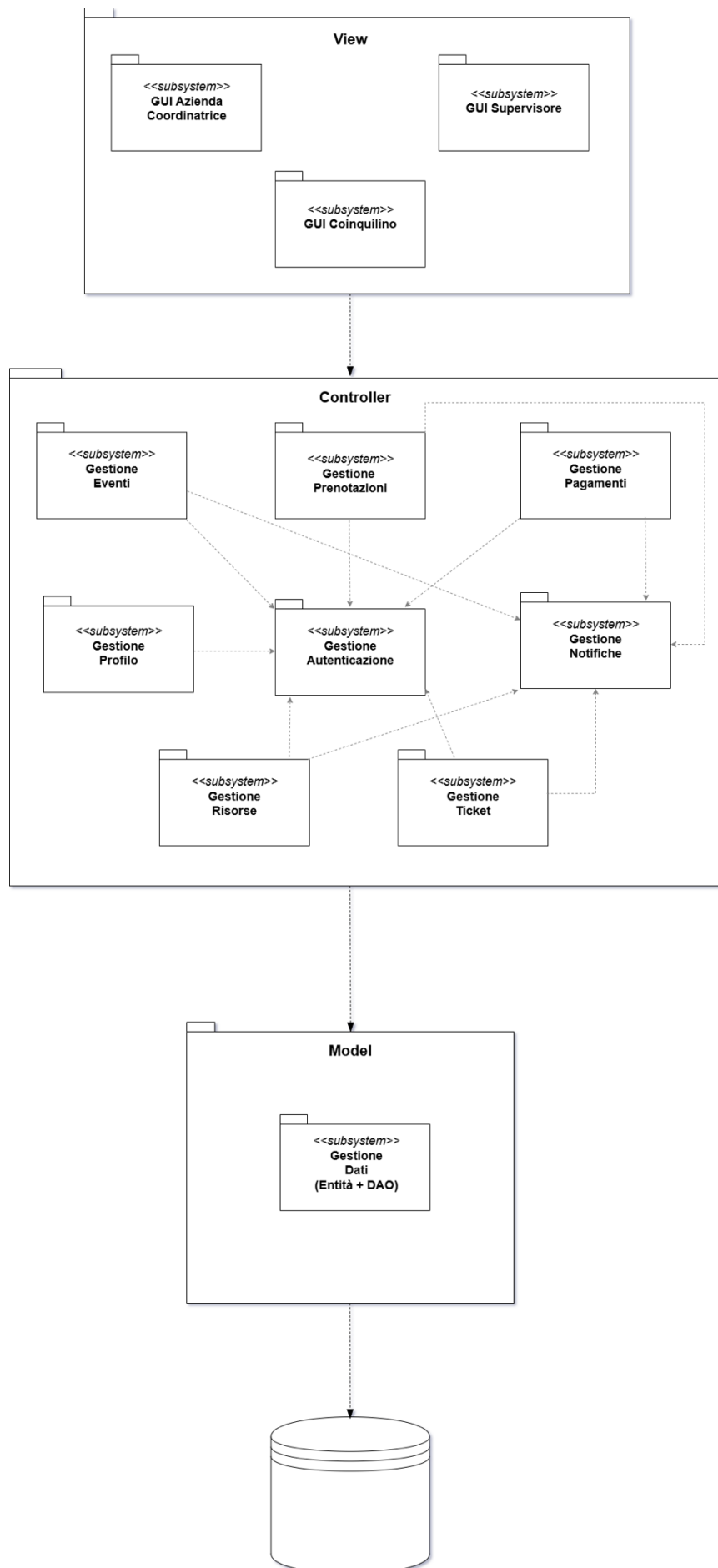
- **Gestione Autenticazione:** Si occupa del login tramite credenziali, della validazione dell'identità dell'utente e della corretta assegnazione dei ruoli.
- **Gestione Profilo Utente:** Gestisce le operazioni relative ai dati personali degli utenti, alla modifica del profilo e alla visualizzazione delle informazioni dell'account.
- **Gestione Prenotazioni:** Si occupa della creazione, modifica, cancellazione e visualizzazione delle prenotazioni degli spazi condivisi. Controlla disponibilità, conflitti di prenotazione e aggiornamento dello stato.

- **Gestione Ticket di Manutenzione:** Gestisce l'intero ciclo di vita dei ticket: creazione, invio, approvazione da parte del supervisore, aggiornamento dello stato e chiusura.
- **Gestione Eventi:** Responsabile della creazione e pubblicazione degli eventi da parte dei supervisori, nonché della gestione delle iscrizioni dei coinquilini.
- **Gestione Risorse Condivise:** Si occupa delle richieste e delle disponibilità delle risorse comunitarie (attrezzature, oggetti condivisi).
- **Gestione Pagamenti e Spese:** Consente la visualizzazione delle spese trimestrali, la conferma dei pagamenti e il controllo delle situazioni economiche da parte dei supervisori.
- **Gestione Notifiche:** Si occupa dell'invio di notifiche automatiche verso gli utenti in base a eventi del sistema (ticket creati, prenotazioni, eventi, pagamenti).

Ciascun sottosistema del Controller realizza uno o più casi d'uso del RAD, garantendo tracciabilità diretta tra requisiti e architettura.

Nella componente Model (Dati e Persistenza):

- **Gestione Dati:** È responsabile del salvataggio e del recupero delle informazioni attraverso il database.
Include le entità principali del sistema (Utente, Comunità, Supervisore, Ticket, Evento, Prenotazione, Pagamento, Risorsa, Notifica) e i rispettivi DAO.



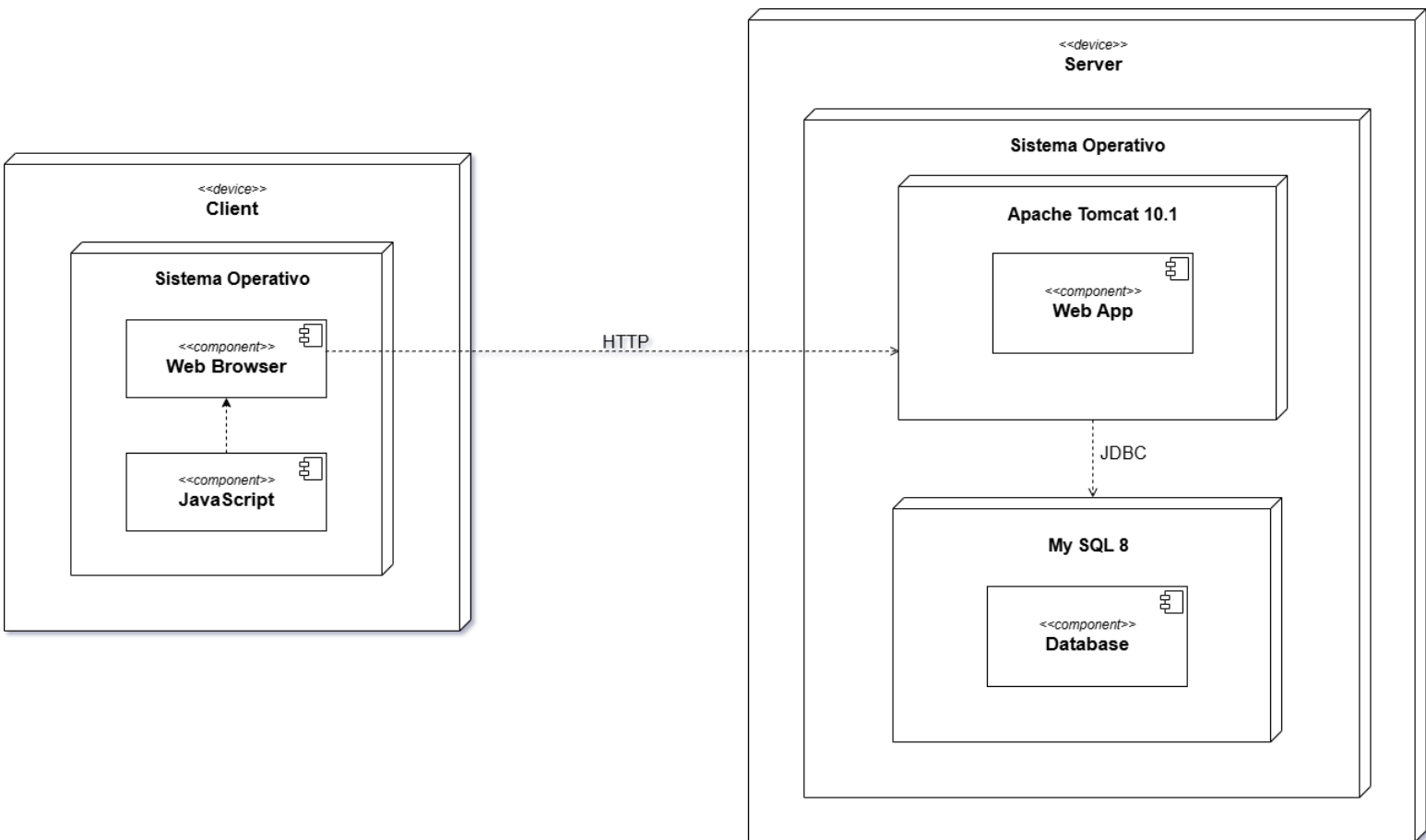
5.3. Hardware/Software mapping

L'architettura di OikoNaos si basa su una web application multilivello distribuita su nodi hardware distinti, con una chiara separazione tra livello di presentazione, livello applicativo e livello di persistenza. Questa organizzazione consente di garantire sicurezza, scalabilità e chiarezza strutturale, in linea con i Design Goals del progetto, in particolare modularità, sicurezza e capacità di crescita futura.

Il **nodo Client** è costituito da qualsiasi dispositivo dotato di un sistema operativo moderno, come Windows, Linux, macOS, Android o iOS, e di un browser compatibile con gli standard HTML5, CSS3 e JavaScript. In conformità con i requisiti di portabilità, l'applicazione è fruibile attraverso Google Chrome, Mozilla Firefox, Microsoft Edge e Apple Safari. Il client non esegue alcuna logica applicativa: il suo compito è esclusivamente quello di presentare l'interfaccia grafica all'utente, interpretare le interazioni e inviare le richieste al server tramite protocollo HTTPS/TLS, assicurando la protezione dei dati trasmessi. Tutta la componente di View descritta nella sezione 5.2 risiede interamente su questo nodo, che rappresenta l'unico elemento eseguito localmente nel browser dell'utente.

Il **nodo Server** costituisce il cuore del sistema ed è incaricato dell'esecuzione dell'intera applicazione web. Esso ospita un application server Apache Tomcat 10.1 e il DBMS MySQL 8, su cui risiede lo schema relazionale progettato a partire dal modello concettuale del RAD. L'applicazione, sviluppata in Java 17, comprende i controller che gestiscono i diversi casi d'uso, le entità del dominio, i servizi che implementano la logica di business e i componenti di accesso ai dati basati sul pattern DAO. In questo nodo risiedono anche i meccanismi di sicurezza fondamentali del sistema: la gestione dei ruoli e dei permessi secondo il modello RBAC, l'hashing delle password, la validazione delle operazioni sensibili e il filtraggio degli input. La comunicazione tra l'applicazione e il database avviene tramite driver JDBC, garantendo affidabilità, integrità e consistenza nelle operazioni di lettura, scrittura e aggiornamento dei dati.

Per motivi di efficienza, affidabilità e semplicità gestionale, si è scelto di collocare l'application server e il database nello stesso nodo fisico. Tale configurazione riduce la latenza tra WebApp e DBMS, semplifica l'infrastruttura e offre un eccellente compromesso tra prestazioni, manutenibilità e chiarezza architetturale. Nonostante ciò, l'architettura rimane pienamente scalabile: qualora in futuro aumentassero il carico o le esigenze prestazionali, sarebbe possibile distribuire i componenti su nodi separati, ad esempio isolando il DBMS o introducendo più istanze Tomcat con bilanciamento del carico, senza modificare la struttura logica del sistema.



5.4. Persistent data management

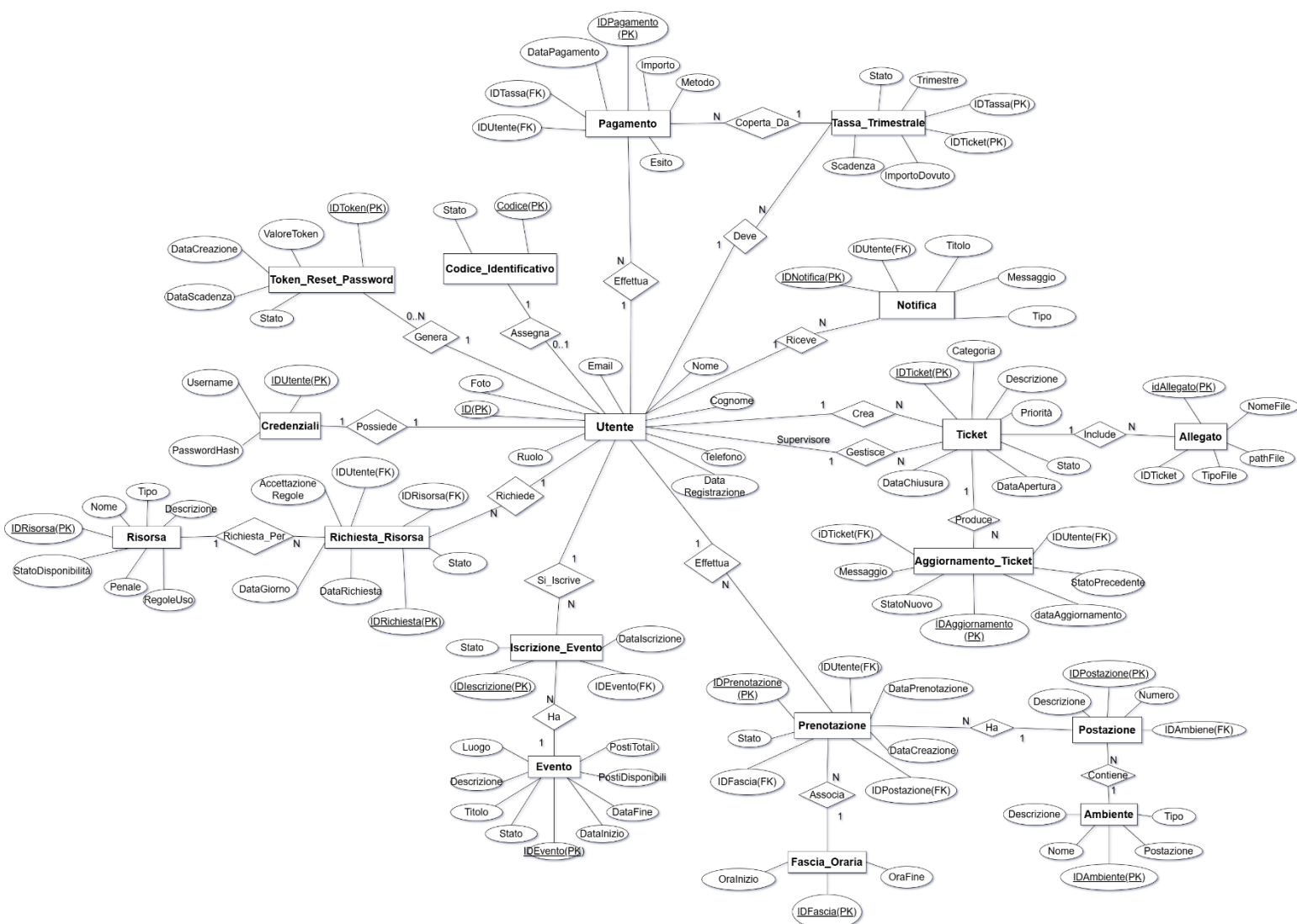
La gestione dei dati persistenti in OikoNaos è affidata al sottosistema Model, che incapsula completamente l'accesso al database e garantisce integrità, sicurezza e coerenza delle informazioni. Il sistema utilizza un database relazionale **MySQL 8**, modellato in accordo con il dominio concettuale definito nel RAD e contenente le principali entità del sistema (Utente, Comunità, Ticket, Evento, Prenotazione, Pagamento, Risorsa, Notifica).

L'accesso ai dati avviene tramite componenti **DAO (Data Access Object)**, che implementano le operazioni CRUD e fungono da intermediari tra la logica applicativa e il DBMS. La comunicazione con il database si basa sul driver JDBC, che permette l'utilizzo di transazioni atomiche per assicurare che operazioni critiche, come la creazione di ticket o l'aggiornamento delle prenotazioni, avvengano sempre in modo consistente.

L'integrità referenziale è garantita tramite vincoli di chiave primaria ed esterna, mentre i controlli applicativi impediscono la generazione di stati incoerenti (es. prenotazioni sovrapposte o pagamento duplicato). I file multimediali, come gli allegati ai ticket, sono invece salvati nel file system del server, con memorizzazione nel database del solo percorso di riferimento.

La sicurezza dei dati è assicurata attraverso hashing delle password (RNF25), sanitizzazione delle query per prevenire SQL injection, e controlli coerenti con il modello RBAC descritto nella sezione 5.5. Sono inoltre previste politiche di backup periodico e procedure di ripristino per garantire continuità operativa.

Il sistema OikoNaos adotta un modello di sicurezza fondato sul controllo rigoroso degli accessi, sulla protezione dei dati persistenti e sulla piena conformità al GDPR. La gestione degli accessi si basa sul modello **Role-Based Access Control (RBAC)**, che distingue tre profili principali: Coinquilino, Supervisore e Azienda Coordinatrice. Ogni ruolo è associato a un insieme circoscritto di permessi, che consente l'accesso alle sole funzionalità pertinenti alle responsabilità dell'utente, garantendo così una netta separazione tra attività comunitarie, amministrative e di supervisione.



5.5. Access control and security

L'autenticazione degli utenti avviene tramite credenziali verificate dal server, con password protette mediante algoritmi di hashing sicuri e conservate nel database in forma non reversibile. Le sessioni

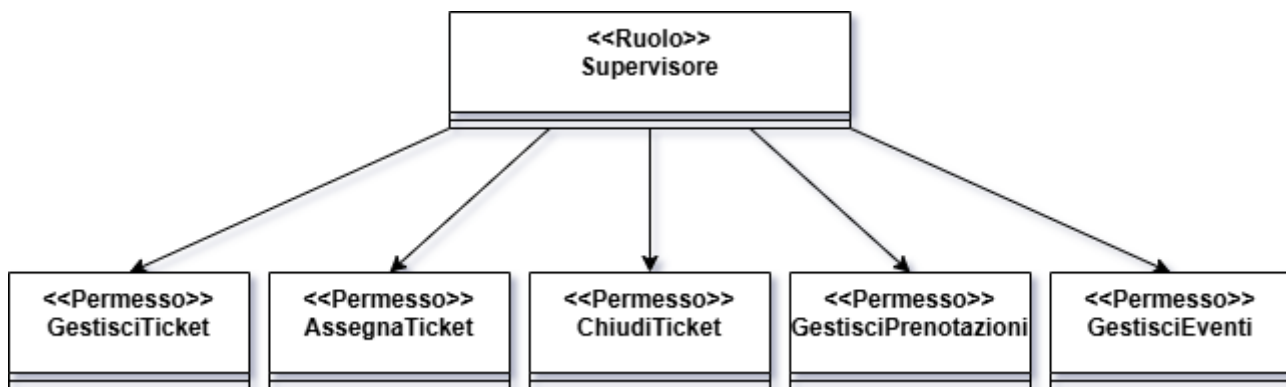
applicative vengono gestite in modo da prevenire utilizzi impropri, accessi non autorizzati e potenziali tentativi di compromissione. Una volta autenticato, l'utente è autorizzato esclusivamente alle operazioni previste dal proprio ruolo: ad esempio, i coinquilini possono aprire ticket, consultare lo stato delle prenotazioni e iscriversi agli eventi; i supervisor possono gestire ticket e prenotazioni, monitorare le risorse condivise e supervisionare le attività della comunità; mentre l'Azienda Coordinatrice può accedere alle funzionalità di amministrazione generale, gestione delle comunità e consultazione delle statistiche.

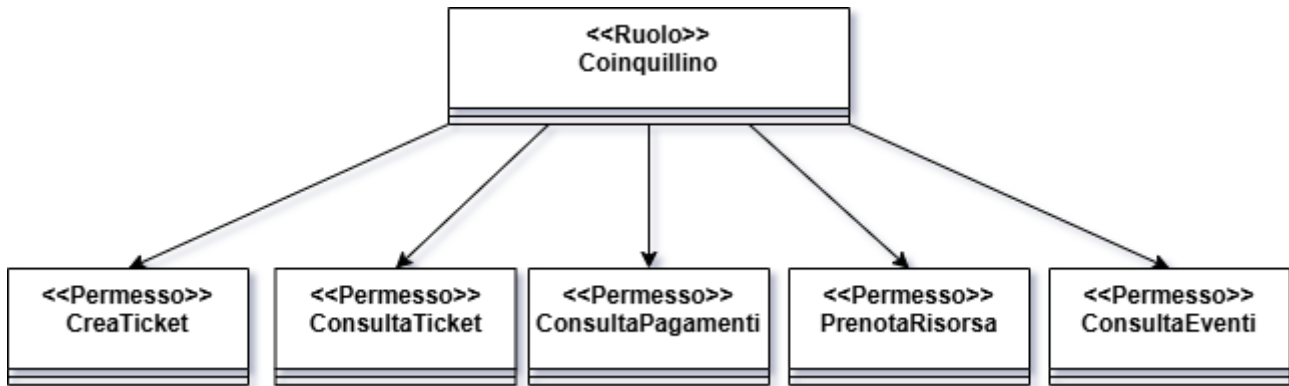
L'autorizzazione è applicata a livello dei singoli sottosistemi e delle relative operazioni, assicurando che ogni richiesta sia convalidata sia rispetto al ruolo dell'utente sia rispetto allo stato del dominio applicativo. A ciò si affiancano ulteriori misure di sicurezza che comprendono la sanitizzazione degli input, la prevenzione delle SQL injection, la gestione corretta dei privilegi del DBMS e l'utilizzo di connessioni protette per l'accesso ai dati. La protezione degli allegati associati ai ticket avviene mediante controlli specifici, che impediscono a utenti non autorizzati di visualizzare o scaricare contenuti sensibili.

Il sistema incorpora inoltre principi di conformità al GDPR, tra cui la minimizzazione e la protezione dei dati personali, l'applicazione di politiche di data retention e la tracciabilità delle operazioni critiche tramite meccanismi di audit. Questo insieme di misure consente a OikoNaos di offrire un ambiente sicuro, resiliente e rispettoso delle normative vigenti, garantendo la tutela delle informazioni degli utenti e la robustezza del sistema nella gestione delle attività di co-housing.

Le figure riportate di seguito rappresentano la struttura RBAC adottata, mostrando l'associazione tra i ruoli del sistema e i rispettivi permessi applicativi.

UML RBAC – Supervisore





5.6. Global software control

Il controllo globale del software in OikoNaos si basa sul modello architetturale tipico degli application server servlet-based, come Apache Tomcat 10.1, il quale gestisce ogni richiesta proveniente dai browser attraverso un pool di thread dedicati. Questo meccanismo consente al sistema di rispondere in modo reattivo a più utenti simultaneamente, mantenendo isolate le richieste e garantendo un'elevata efficienza anche in condizioni di carico elevato.

Il comportamento complessivo del sistema segue l'architettura Model–View–Controller (MVC). Quando un utente interagisce con l'interfaccia web, la View invia una richiesta HTTPS al server. Tomcat intercetta la richiesta e la assegna a un thread disponibile. Il controller appropriato interpreta il caso d'uso da eseguire e coordina le operazioni applicative necessarie, dialogando quando serve con il livello Model, che incapsula le entità del dominio e i componenti di accesso ai dati. Una volta completata l'elaborazione, il controller affida nuovamente il controllo alla View, la quale genera la risposta destinata al browser dell'utente.

La **gestione della concorrenza** rappresenta uno degli aspetti più delicati del flusso di controllo. Operazioni come la creazione o modifica di prenotazioni, l'aggiornamento dello stato dei ticket o la registrazione dei pagamenti coinvolgono dati condivisi e potenzialmente soggetti a conflitti. Per questo motivo, OikoNaos combina meccanismi provenienti da più livelli: Tomcat gestisce la concorrenza tramite i thread, l'applicazione applica validazioni e controlli sullo stato delle risorse prima di ogni modifica, mentre il DBMS garantisce l'integrità dei dati attraverso transazioni atomiche e lock impliciti. In questo modo si evitano condizioni incoerenti, come prenotazioni sovrapposte o aggiornamenti concorrenti sullo stesso ticket.

La **gestione degli errori e dei fallimenti** segue un approccio strutturato. Eventuali anomalie emerse durante l'elaborazione di una richiesta vengono intercettate dal controller attraverso eccezioni controllate; in presenza di operazioni critiche non portate a termine con successo, il sistema effettua un rollback automatico delle transazioni, prevenendo stati parziali o inconsistenti. Le informazioni sugli errori e sugli eventi rilevanti vengono registrate nel servizio di Logging & Audit, che supporta le attività di diagnosi e il monitoraggio del comportamento del sistema, senza esporre all'utente dettagli interni o informazioni sensibili.

Grazie alla combinazione del modello thread-driven di Tomcat, della chiara separazione tra View, Controller e Model e dell'uso coordinato di transazioni, validazioni e controlli di integrità, OikoNaos garantisce un flusso di controllo stabile, sicuro e affidabile. Ciò consente al sistema di operare correttamente in contesti altamente concorrenti, come quelli tipici delle comunità di co-housing, assicurando continuità di servizio e coerenza dei dati anche in presenza di numerosi utenti attivi simultaneamente.

5.7. Boundary conditions

Le Boundary Conditions definiscono le condizioni iniziali, finali ed eccezionali che il sistema OikoNaos deve gestire per garantire stabilità, continuità operativa e sicurezza dei dati. Esse comprendono le precondizioni necessarie affinché un'operazione possa essere eseguita correttamente, le postcondizioni garantite al termine dell'elaborazione e i meccanismi di gestione degli errori o dei guasti che possono verificarsi durante l'esecuzione.

Le **precondizioni** riguardano, ad esempio, la presenza di un utente autenticato, la validità del codice identificativo durante la registrazione, la disponibilità di una risorsa prima di una prenotazione o la correttezza dei parametri passati ai controller applicativi. Le **postcondizioni** includono invece la persistenza atomica dei dati, la coerenza tra le entità del dominio, la conferma dell'operazione attraverso l'interfaccia utente e la generazione di eventuali notifiche interne. Il sistema garantisce che, in caso di completamento con successo, ogni operazione lasci il dominio in uno stato consistente e verificabile.

Durante l'esecuzione il sistema può incontrare condizioni di errore dovute a input non validi, credenziali scorrette, collisioni nelle prenotazioni, tentativi di accesso non autorizzato, perdita temporanea di connettività o anomalie nei servizi applicativi. In tali situazioni, OikoNaos applica **strategie di mitigazione** che comprendono messaggi di errore chiari e non tecnici, rollback delle transazioni per evitare stati parziali, ripetizione controllata dell'operazione e, quando necessario, reindirizzamento verso pagine dedicate alla gestione delle anomalie. La protezione dell'integrità dei dati è garantita sia dal livello applicativo, attraverso controlli sullo stato del dominio, sia dal DBMS, tramite transazioni atomiche e lock impliciti.

Sono previste anche **condizioni eccezionali** di livello infrastrutturale, come guasti del server, arresti improvvisi di Tomcat, indisponibilità del DBMS MySQL, corruzione degli allegati o interruzioni dell'alimentazione elettrica. Il sistema reagisce attraverso procedure strutturate: in caso di failure dell'application server, l'accesso viene bloccato e viene restituito un messaggio di servizio non disponibile; in caso di indisponibilità del database, tutte le operazioni vengono sospese e viene impedita la modifica del dominio fino al ripristino; quando si verificano eccezioni non previste, l'applicazione registra l'evento nel servizio di Logging & Audit e produce una risposta generica, evitando di esporre dettagli interni.

Per garantire resilienza, l'infrastruttura può adottare **configurazioni hardware ridondate**, come dischi in RAID o percorsi di rete alternativi. In caso di interruzioni di corrente, l'utilizzo di UPS permette al server di completare le operazioni in corso o di eseguire un arresto controllato. Al riavvio, il sistema ricostruisce lo stato tramite log e meccanismi di recovery transazionale, assicurando che il dominio rimanga consistente.

Dal punto di vista operativo, OikoNaos prevede **procedure formali** di start-up e shut-down, eseguite da un amministratore di sistema. L'inizializzazione richiede l'avvio del DBMS MySQL, del server Apache Tomcat e il deploy della WebApp; la terminazione segue un ordine inverso, con chiusura controllata delle sessioni e completamento delle operazioni pendenti. In caso di failure durante l'avvio, come l'assenza del DBMS o l'arresto imprevisto di Tomcat, l'applicazione blocca l'accesso e segnala l'indisponibilità del servizio.

Infine, alcune operazioni amministrative non rientrano nelle funzionalità direttamente disponibili all'interno della WebApp. L'inserimento o l'eliminazione di un account supervisore, ad esempio, viene

eseguito manualmente da un operatore dell'Azienda Coordinatrice direttamente sul database MySQL, mediante strumenti di amministrazione o riga di comando. Questa scelta progettuale garantisce un maggiore controllo sulle identità dotate di privilegi elevati ed è documentata attraverso un apposito caso d'uso amministrativo incluso nel RAD.

6. Subsystem Service Glossary

Questa sezione raccoglie e descrive in modo sintetico tutti i servizi offerti dai sottosistemi logici di OikoNaos. Ogni sottosistema del livello *Controller* espone un insieme di operazioni che implementano i casi d'uso definiti nel RAD, mentre il livello *Model* fornisce servizi di persistenza e gestione delle entità del dominio.

Le seguenti tabelle riassumono i servizi principali, raggruppati per sottosistema.

Gestione Autenticazione

<i>Servizio</i>	<i>Descrizione</i>
Autenticazione	Verifica credenziali (username e password) e avvia una sessione autenticata.
Logout	Termina la sessione dell'utente.
Recupero password	Genera un token e gestisce la procedura di recupero delle credenziali.
Validazione codice identificativo	Verifica la validità del codice fornito in fase di prima registrazione.
Identificazione ruolo	Associa l'utente autenticato al ruolo corretto (coinquilino, supervisore, azienda).

Gestione Profilo Utente

<i>Servizio</i>	<i>Descrizione</i>
Visualizza profilo	Recupera i dati anagrafici e di contatto dell'utente.
Modifica informazioni	Consente l'aggiornamento del profilo (foto, telefono, e-mail).
Modifica credenziali	Permette di aggiornare la password in conformità ai requisiti di sicurezza.
Eliminazione account	Rimuove definitivamente l'account dell'utente dal sistema.

Gestione Prenotazione

<i>Servizio</i>	<i>Descrizione</i>
Creazione prenotazione	Permette la prenotazione di una postazione per una certa data e fascia oraria.
Modifica prenotazione	Aggiorna data, orario o postazione, verificando disponibilità e conflitti.
Cancellazione prenotazione	Elimina una prenotazione esistente.
Verifica disponibilità	Controlla se la postazione è disponibile nella fascia richiesta.
Visualizza prenotazioni	Mostra lo storico e le prenotazioni attive dell'utente.
Approvazione prenotazioni	Permette al supervisore di approvare o rifiutare una prenotazione.

Gestione Ticket di Manutenzione

Servizio	Descrizione
Creazione ticket	L'utente può segnalare un guasto indicando descrizione, categoria e priorità.
Allegazione file	Permette di associare immagini o documenti alla segnalazione.
Aggiornamento ticket	Il supervisore può aggiornare stato, priorità o assegnazione.
Chiusura ticket	Termina il ciclo di vita del ticket e registra lo stato finale.
Visualizza ticket	Permette agli utenti di consultare i ticket creati e il loro stato.
Storico aggiornamenti	Mostra tutte le modifiche eseguite sui ticket.

Gestione Eventi

Servizio	Descrizione
Creazione evento	Il supervisore crea un evento con luogo, data e posti disponibili.
Pubblicazione evento	L'evento viene reso visibile nella bacheca.
Iscrizione evento	Il coinquilino può iscriversi, diminuendo i posti disponibili.
Visualizza eventi	Mostra eventi futuri e passati.
Gestione posti disponibili	Controlla, aumenta o riduce i posti disponibili.

Gestione Risorse Condivise

Servizio	Descrizione
Richiesta risorsa	L'utente inoltra una richiesta per usufruire di una risorsa condivisa.
Accettazione regole d'uso	Registra l'accettazione delle condizioni di utilizzo.
Gestione disponibilità	Aggiorna lo stato della risorsa (disponibile/non disponibile).
Visualizza risorse	Mostra tutte le risorse disponibili e quelle occupate.
Gestione penalità	Applica penalità secondo le regole della comunità.

Gestione Pagamenti

Servizio	Descrizione
Visualizzazione spese	Mostra l'importo della tassa trimestrale e le spese associate.
Registrazione pagamento	Consente al sistema di marcare una tassa come pagata.
Verifica stato pagamento	Controlla lo stato (pagato, in attesa, scaduto).
Aggiornamento importi	Aggiorna automaticamente le tasse trimestrali.
Consultazione storico	Mostra i pagamenti passati e le ricevute.

Gestione Notifiche

Servizio	Descrizione
Generazione notifica	Produce notifiche interne in seguito a eventi rilevanti.
Invio notifiche	Invia notifiche agli utenti interessati.
Visualizza notifiche	Mostra le notifiche non lette e lo storico.
Aggiornamento stato notifica	Segna le notifiche come lette o visualizzate.

Sottosistema Model – Persistenza Dati

<i>Servizio</i>	<i>Descrizione</i>
CRUD entità	Operazioni di creazione, lettura, aggiornamento e cancellazione su tutte le entità del dominio.
Gestione transazioni	Garantisce atomicità e consistenza durante operazioni critiche.
Recupero dati	Fornisce dati filtrati a supporto dei casi d'uso (eventi, ticket, prenotazioni, risorse).
Gestione allegati	Salva e recupera file associati ai ticket.
Vincoli di integrità	Applica regole relazionali per garantire consistenza della base dati.